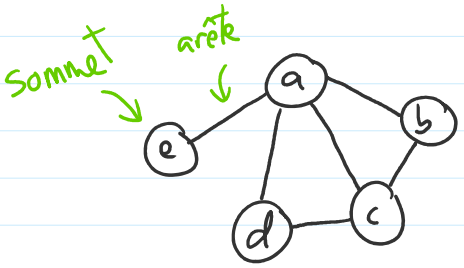
Graphes

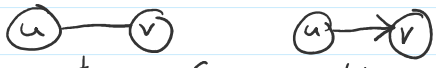
25 novembre 2024 16:55

- Graphes = objets + liens

  = sommets + arêtes

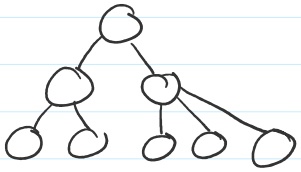


- sommets/arêtes peuvent avoir un poids
- arêtes peut être dirigées ou pas



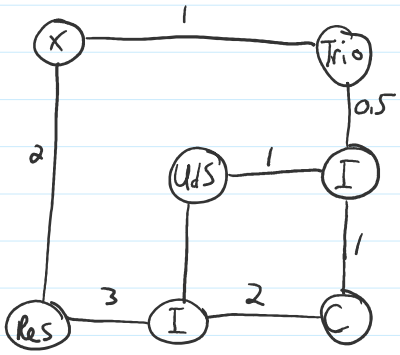
- Sommets parfois appelés noeuds

Arbre = cas spécial de graphe



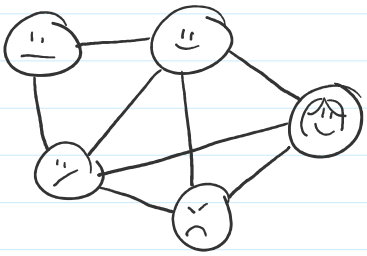
- Applications
  - Maps : sommet = endroit
    arête = route
    (poids = dist)

    souvent orienté

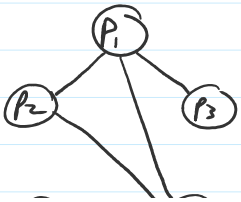


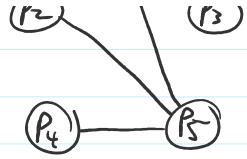
  - Réseau social

    sommet = personne
    lien = ami



  - Bio-info : réseau d'interaction de protéines





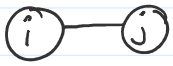
- etc etc

Notation $G = (V, E)$

graphe — vertex (sommets) — edge (arêtes)

V = ensemble $\qquad$ E = ensemble de paires d'elts de V

Souvent, $V = \{0, 1, 2, \ldots, n-1\}$



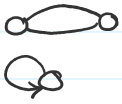
- G non-orienté $\rightarrow$ E contient des paires non-ordonnées $\{i, j\}$

  orienté $\rightarrow$ paires ordonnées

(i, j)

- Si non-spécifié, par défaut

  G est non-orienté
  pas d'arête répétée
  pas d'arête vers soi-même

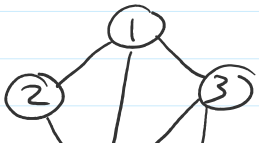


- j est un <u>voisin</u> de i si $\{i, j\}$ forme une arête.

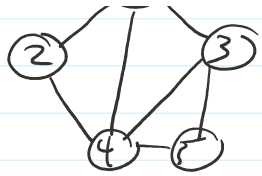
  Façon succincte d'écrire $\{i, j\} = ij$

- Degré de i = nb de voisins de i

  $\deg(i) = |\{j : ij \in E\}|$

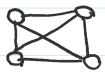
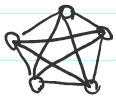


sommet	degré
1	3
2	2
3	3



1	3
2	2
3	3
4	4
5	2

Q: si G a n sommets, combien a-t-il d'arêtes, au maximum?

	#sommets	#max arêtes
	1	0
	2	1
	3	3
	4	6
	5	10
	6	15
	7	26
	$\vdots$	
	n	$\frac{n(n-1)}{2} = O(n^2)$

Au max, chaque i a $n-1$ voisins

$\Rightarrow \frac{n \cdot (n-1)}{2}$ arêtes

($\div 2$ car chaque arête comptée 2 fois)

Proposition: la somme des degrés est égale à

$\#\text{arêtes}/2 = m/2$

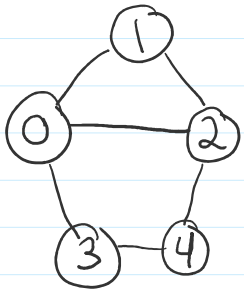
$$\deg(0) + \deg(1) + \deg(2) + \cdots + \deg(n-1) = m/2$$

Représentations d'un graphe

1) Liste d'adjacence

Liste de liste adj

adj[0] = voisins de 0
adj[1] = voisins de 1
⋮
adj[n-1] = voisins de n-1



adj

[0] → [1,2]
[1] → [0,2]
[2] → [1, 0, 4]
[3] → [4,0]
[4] → [2,3]

Plusieurs façons

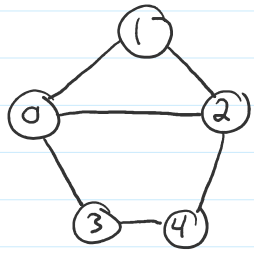
```
list<list<int>> adj;

vector<vector<int>> adj;

map<int, unordered_set<int>> adj;
```

...

2) Matrice d'adjacence

- Tableau 2D $n \times n$ int** tab
- $tab[i][j] = \begin{cases} 1 & \text{si } i,j \text{ forment une arête} \\ 0 & \text{sinon} \end{cases}$

- Si non-orienté, $tab[i][j] = tab[j][i]$



	0	1	2	3	4
0	0	1	1	1	0
1	1	0	1	0	0
2	1	1	0	0	1
3	1	0	0	0	1
4	0	0	1	1	0

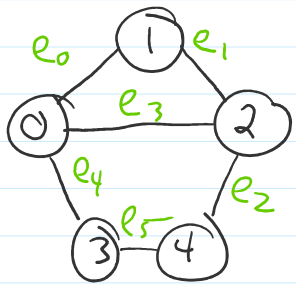
3) Matrice d'incidence

- Si n sommets, m arêtes,

  tableau 2D $n \times m$ — n rangées, m colonnes

- Chaque arête numérotée $e_0, e_1, \dots, e_{m-1}$

$$tab[i][p] = \begin{cases} 1 & \text{si } i \text{ est un bout de } e_p \\ 0 & \text{sinon} \end{cases}$$

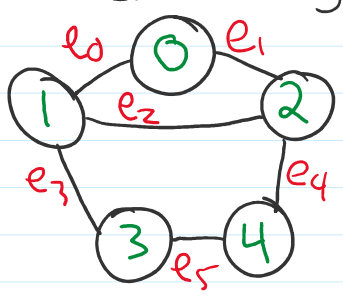


	e_0	e_1	e_2	e_3	e_4	e_5
0	1	0	0	1	1	0
1	1	1	0	0	0	0
2	0	1	1	1	0	0
3	0	0	0	0	1	1
4	0	0	1	0	0	1

TDA : Graphe

- ajouter un sommet / une arête
- supprimer un sommet (et ses arêtes incidentes)
- supprimer une arête

- supprimer une arête
- énumérer les voisins d'un sommet
- obtenir le degré
- savoir si i,j est une arête



1) Liste adj vector<vector<int>>

0 →	1	2	
1 →	0	2	3
2 →	1	4	0
3 →	4	1	
4 →	3	2	

2) Mat. adj

	0	1	2	3	4
0	0	1	1	0	0
1	1	0	1	1	0
2	1	1	0	1	0
3	0	1	1	0	1
4	0	0	0	1	0

3) Mat. incidence

	e_0	e_1	e_2	e_3	e_4	e_5
0	1	1	0	0	0	0
1	1	0	1	1	0	0
2	0	1	1	0	1	0
3	0	0	0	1	0	1
4	0	0	0	0	1	1

Complexités

	Liste adj	Mat adj	Mat incidence
Ajouter un sommet	$O(1)$ amorti adj.push_back	$O(n^2)$ Redim	$O(mn)$ Redim
Verifier si $\{i,j\}$ est une arête	$O(\min(\deg(i),\deg(j)))$ Parcourir adj[i] ou adj[j], celle avec le size() min	$O(1)$ return tab[i][j]	$O(m)$ Pour chaque colonne c si tab[i][c]==1 et Tab[j][c]==1 return true
Ajouter une arête	$O(\min(\deg(i),\deg(j)))$	$O(1)$	$O(nm)$ Redim

Ajouter une arête	$O(\min(\deg(i),\deg(j)))$ Vérifier, puis 2× push_back dans adj[i], adj[j]	$O(1)$ Tab[i][j]=1	$O(nm)$ Redim
Supprimer une arête {i,j}	$O(\deg(i)+\deg(j))$ erase dans adj[i] et adj[j]	$O(1)$ tab[i][j]=0	$O(nm)$ Redim
Énumérer les voisins de i	$O(\deg(i))$ Itérer adj[i]	$O(n)$ Itérer rangée i	$O(m+\deg(i)\cdot n)$ m [Pour chaque colonne c $\deg(i)\cdot n$ [si tab[i][c]==1 itérer colonne c
Supprimer un sommet i	$O(m+n)$ Pour chaque j adj[j].erase(i) adj[i].clear() Temps $= O(\deg(0)+\dots+\deg(n)) = O(2m) = O(m)$	$O(n)$ Mettre 0 sur col i / rangée i	$O(nm)$ Redim

Version simple: on laisse i mais on supprime toutes ses arêtes

mieux = parcourir voisins de i

Pour k=0..adj[i].size()-1

adj[adj[i][k]].erase(i)

adj[i].clear()

Temps = $O(d)$ où

d = somme des degrés des voisins de i $= \sum_{j \text{ voisin de } i} \deg(j)$